# AGENTS.md

> `AGENTS.md` is one of the most important files in an AI assisted project.

---

It is not written for users. It is not the same as your README. It is written for the AI agent that will help build the application.

The purpose of this file is simple. When the AI starts working, it should already understand the product, the stack, the rules, the workflow, and the boundaries of the project.

For this learning platform, `AGENTS.md` explains what the app is, how the AI should work, how UI should be implemented, how content is modeled, how search behaves, how videos are processed, what must stay private, and what checks need to run.

**That is why this file matters. It gives the AI a system to follow instead of leaving every decision open during implementation.**

---

## Start with the mindset

Before writing `AGENTS.md`, understand the role split clearly. You are the product thinker. The AI is the implementation agent.

| YOU | AI |
| --- | --- |
| **You are the product thinker.** | **The AI is the implementation agent.** |

The AI can move fast, but it should not be responsible for deciding what the product is, what the architecture should be, or what tradeoffs matter. That thinking belongs to you.

Your job is to define the product, the stack, the boundaries, the rules, and the workflow. The AI's job is to execute inside that system.

A weak starting prompt looks like this.

```
- Build me a learning platform with AI search.
```

That gives the AI too much freedom. It may choose the wrong data model, build the wrong search experience, expose private values, create routes you did not want, or add features that are outside the current build.

A stronger starting point looks like this.

> You are building a learning platform where authors create courses in Sanity and learners browse, watch lessons, and search across course content. Search must be grounded in Sanity data, return structured result cards, and link to exact timestamps in lesson videos. Before implementation, write a prompt file, ask for approval, then build strictly to that approved plan.

That is the difference. You are not asking the AI to design the whole system during implementation. You are giving it the system first.

**`AGENTS.md` is where that system lives.**

---

## Think before writing the file

Do not open a blank `AGENTS.md` file and start adding random rules. First, clarify the product.

For a learning platform, start with basic product questions. What does the app do. Who creates the content. Who consumes the content. Where does the content live. What pages does the learner use. What parts are public. What parts require authentication.

Then go deeper. What makes this product different. What should search return. What should search never invent. What data should stay private. What should not be built yet.

For this project, the product is clear. Authors create course content in Sanity, and learners use a Next.js app to browse courses, open lessons, watch videos, and search across learning content.

The standout feature is intelligent search. A learner should be able to type a natural language query and get useful result cards that point to real courses, real lessons, or exact moments inside lesson videos.

That one product decision affects almost everything else. It affects the Sanity schema, the video ingestion pipeline, the search API, the result cards, the timestamp behavior, and the server and client responsibilities.

So before writing `AGENTS.md`, get the product clear. Then write the instructions.

---

## Treat AGENTS.md as the project operating manual

A good `AGENTS.md` is not just a list of preferences. It is the disciplined manual for how the AI should work inside the codebase.

For this learning platform, the file should answer the important questions. What are we building. How should the AI work. How should UI work be handled. Which skills should the AI read. What stack are we using. What data are we modeling. How does search behave. What mistakes should the AI avoid. What checks must run.

This keeps the AI consistent across the whole build. If it forgets whether search should be a chatbox or result cards, `AGENTS.md` answers that. If it tries to expose a private value in the browser, `AGENTS.md` says no. If it tries to build a custom video player, `AGENTS.md` says no.

The point is not to make the file long for no reason. The point is to remove decisions the AI should not be making during feature implementation.

SECTION 1

## Define the AI role

Start by telling the AI who it is. Do not write something vague like this.

```
- You are a helpful coding assistant.
```

That does not tell the agent how seriously to treat architecture, security, product behavior, or testing.

For this learning platform, the AI should act like a senior implementation partner.

```
You are a principal level full stack engineer and AI implementation agent building a production style AI powered learning platform with intelligent content search.
```

This gives the AI the right posture. It should not behave like a casual code generator. It should think about structure, security, implementation quality, data flow, and whether the result actually matches the product.

Then define the core job.

```
Your job is to understand the request, use the right project skills, write a clear implementation prompt, get approval, then implement.
```

That one sentence is important because it tells the AI that coding is not the first step. Planning is.

SECTION 2

## Explain what you are building

This section should describe the product in plain language. The AI needs to understand the app as a product, not just as a collection of technologies.

For the learning platform, you can explain it like this.

```
This is a learning platform. Authors create courses in Sanity, and a Next.js app serves those courses to learners. Learners can browse courses, open course pages, watch lessons, and search across the learning content.
```

Then explain what makes the app special.

```
The standout feature is intelligent search. A learner types a plain language query and gets ranked clickable result cards. Those cards can link to courses, lessons, or exact seconds in lesson videos where the topic is taught.
```

This matters because the AI needs to understand the product goal behind the feature list. If it understands that search is the standout feature, it will make better choices when working on schemas, transcripts, search routes, result cards, and timestamp playback.

After that, list what is in scope in normal language. For this project, that includes the Sanity content model, authentication, catalog page, course details page, lesson page, instructor pages, learner progress, PostHog analytics, transcript ingestion, search configuration, and the search experience.

Then add the scope boundary clearly.

```
Build nothing beyond that. Do not overbuild.
```

This is short, but important. AI agents often add extra features because they seem useful. Your file should make it clear that useful is not enough. The feature must be in scope.

---

SECTION 3

## Define how the AI should work

This is one of the most important sections in the whole file. The AI should not open the codebase and immediately start editing.

For this project, the workflow should be very clear.

| | |
|---|---|
| 01 | Read AGENTS.md. |
| 02 | Read the named skills and any supporting skills needed. |
| 03 | Inspect existing code and config. |
| 04 | Ask one focused question only if the task is genuinely ambiguous. |
| 05 | Write an implementation prompt in prompts. |
| 06 | Ask for approval. |
| 07 | Build only after approval. |
| 08 | Run checks. |
| 09 | Close with a short report. |

This workflow slows the AI down in the right way. It prevents the agent from jumping into implementation before it understands the request, the existing code, or the boundaries of the task.

The implementation prompt should include the goal, skills read, code inspected, decisions and assumptions, expected files, requirements, security considerations, acceptance criteria, checks to run, and manual test steps.

That prompt file becomes the plan. You review the plan before the AI touches the code.

The workflow is simple.

PLAN → REVIEW → APPROVE → IMPLEMENT → TEST → FIX → SHIP

After implementation, the AI should close with three short sections.

| 01 | 02 | 03 |
| --- | --- | --- |
| **What I did** | **Test** | **Needs your attention** |

The final report should be short. The detailed reasoning belongs in the prompt file, not in a long message after implementation.

---

SECTION 4

## Define the UI rules

AI is not the designer in this workflow. You should either provide UI screenshots or integrate it with Figma MCP, and the AI implements them.

That needs to be explicit because AI often tries to improve a design. It may change colors, spacing, typography, layout, shadows, or component structure because it thinks the result looks better.

For this learning platform, the UI rule should be direct.

```
You do not design UI. The user gives desktop images plus a prompt.
Reproduce the reference exactly including layout, spacing, typography,
color, and states.
```

Then explain mobile behavior.

```
There is no mobile reference, so make the page responsive sensibly
while keeping the desktop reference exact.
```

This gives the AI the right balance. Desktop should match the screenshot. Mobile should adapt cleanly because no mobile screenshot exists.

Also tell the AI to reuse existing components and Tailwind patterns before adding new ones. This keeps the app consistent across pages and prevents every feature from introducing a new visual style.

The most important rule is this.

```
When there is a reference image, it is the source
of truth.
```

That prevents the AI from treating the screenshot as a loose suggestion.

---

SECTION 5

## Tell the AI which skills and docs to use

The AI should not rely only on memory for every technology. If the project has installed skills, local docs, or package docs, list them.

For this learning platform, useful skills include the Sanity best practices skill, the Sanity migration skill, the Sanity Context setup skill, the context tuning skill, the agent shaping skill, and the local Next.js docs inside `node_modules` (which will be already mentioned by Next.js in `AGENTS.md`)

Do not just list names. Explain when the AI should use them.

| | |
|---|---|
| 01<br>The Sanity best practices skill should be used for schemas, GROQ, Portable Text, TypeGen, and framework integration. | 02<br>The Sanity migration skill should be used for importing content. |
| 03<br>The Sanity Context setup skill should be used when wiring search to the Context MCP. | 04<br>The context tuning skill helps define what content the search agent can see and how the Context document should guide it. |
| 05<br>The agent shaping skill helps define how the search agent behaves, what it should return, and what it must avoid. | 06<br>Also tell the AI to read local Next.js docs before writing Next.js code. Framework conventions change, and local docs are safer than memory. |

This section is about reducing uncertainty. The AI should use the project's actual tools, not whatever it remembers from older examples.

---

SECTION 6

## Explain app responsibilities

This section tells the AI where responsibilities belong. It protects the project from messy code and unsafe shortcuts.

For this learning platform, explain the main responsibilities clearly.

- The learner facing app displays stored content.
- Sanity Studio is for content authoring.
- Clerk handles authentication.
- PostHog captures product analytics.
- The search API runs on the server.
- The browser renders UI and calls safe app routes.
- The video ingestion pipeline runs as offline tooling.

The most important boundary is between browser and server.

- The browser should not hold private tokens.
- It should not call the LLM directly.
- It should not call the Sanity Context MCP directly.
- It should not write content or progress directly.

A simple rule is this.

> The browser only shows UI and calls safe app routes. Private data access and AI calls happen on the server.

For this project, that protects Sanity read tokens, Sanity write tokens, Clerk secret keys, PostHog private keys, OpenAI keys, and Sanity Context credentials.

This section prevents some of the most dangerous AI mistakes. A feature can look like it works and still be wrong if it exposes secrets or moves server work into the browser.

---

SECTION 7

## List the tech stack

Do not assume the AI knows the stack. Write it clearly.

For this learning platform, the stack includes Next.js App Router, TypeScript, Tailwind, Sanity Studio, `next-sanity`, `@sanity/image-url`, `@portabletext/react`, Clerk, PostHog, Sanity Context MCP, Vercel AI SDK, OpenAI, Zod, and `react-markdown` only for search reply rendering.

This section helps the AI choose the right tools when implementing features. It also prevents unnecessary additions.

- You should also list what not to use. For example, do not use a public dataset for private content, do not put tokens in client components, do not add a separate backend framework, do not use Sanity auth instead of Clerk, and do not use semantic similarity unless embeddings are enabled.

Negative instructions are important. AI agents often choose reasonable looking alternatives. Your job is to make it clear which alternatives are not allowed for this project.

---

SECTION 8

## Record decisions already made

This section prevents the AI from deciding the same product questions again during every feature.

For the learning platform, decisions include the following.

| | |
|---|---|
| 01 Search is a full results page, not a chatbox. | 02 Search results are structured cards. |
| 03 Search is grounded in Sanity data. | 04 Video intelligence lives in video documents. |
| 05 Timestamps resolve chapters first and transcript chunks second. | 06 Playback stays on the site through provider embeds. |

The data decisions matter too.

- Modules are embedded inside courses.
- Lessons are separate documents.
- Lesson numbers are derived from order.
- Progress is keyed by Clerk user id.
- PostHog tracks meaningful learning events.

This section matters because the AI may try to be creative. It might build chat instead of result cards, create a module document, store parent course directly on lessons, show raw video documents as results, or send learners away to YouTube.

The decisions section tells the AI that these choices are already made. Build to them unless the user changes them.

---

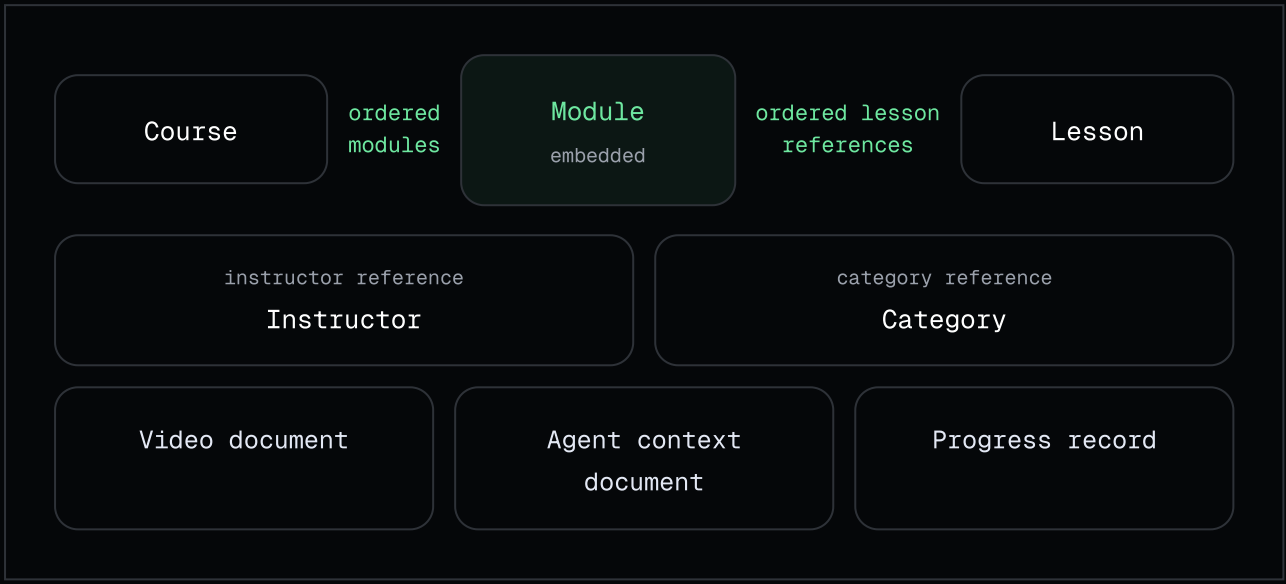
SECTION 9

## Define the data model

For a learning platform, the data model is critical. The app is only as good as the structure of its content.

Search quality depends on content quality. Course pages, lesson pages, transcript matching, and timestamp results all depend on data being modeled clearly.

So `AGENTS.md` should define the main documents and records.




## Course

A course is the main thing learners browse. It should include a title, slug, summary, cover image, level, price, optional popular flag, student count, learning outcomes, instructor reference, category reference, and ordered modules.

The course controls the learning path. That means the order of modules and lessons belongs here.

## Module

A module is embedded inside a course. It should not be a top level document.

This is important because a module only makes sense within a specific course. A module called “Getting Started” or “Advanced Concepts” is not useful by itself. It is useful because it belongs to a course and contains lessons in a specific order.

A module should include a title, summary, and ordered lesson references.

## Lesson

A lesson is a separate document because it has content of its own. It should include a title, slug, video URL, thumbnail or poster, duration, free preview flag, student count, Portable Text notes, key points, optional pro tip, and resources.

A lesson does not need to store its parent course. When the app needs the parent course, it can derive it through the course's lesson references.

## Instructor

An instructor should include a name, slug, photo, expertise, and bio. Instructors can appear on course pages, lesson pages, and their own instructor pages later.

## Category

A category should include a title, slug, and description. Categories help with browsing, filtering, and course organization.

## Video document

A video document is created by the ingestion pipeline. It should include a video id, video URL, chapters with `startSeconds` and labels, and transcript chunks with `startSeconds` and text.

- Do not store the whole transcript in one field that gets returned wholesale. That makes search noisy, expensive, and harder to control.

### Agent context document

The agent context document stores search configuration. It should include a content scope filter and search instructions.

This lets search behavior be tuned without changing application code every time.

### Progress record

Progress belongs to app state, not content. It should be keyed by Clerk user id and store completed lessons plus the learner's last position in a lesson.

Progress writes should go through a server route. The browser should not write progress directly.

---

SECTION 10

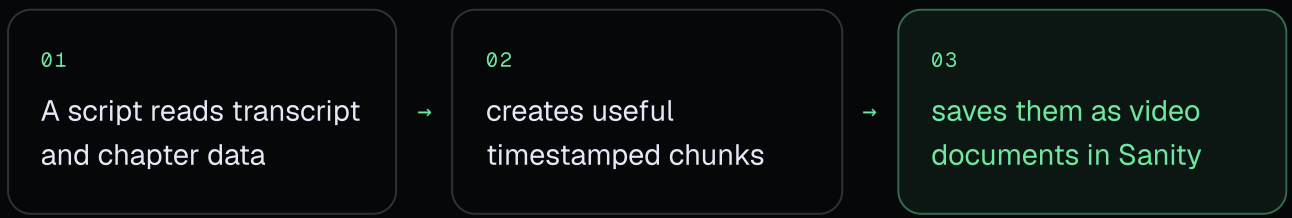
## Explain video ingestion

A lesson has a video URL, but a video URL alone is not searchable. If a learner asks where Clerk middleware is explained, the app needs to know what is said inside the video.

That is why the project needs video ingestion.

Video ingestion is an offline process. Offline does not mean disconnected from the internet. It means the work happens outside the normal learner request.

A script reads transcript and chapter data, creates useful timestamped chunks, and saves them as video documents in Sanity. Later, search can use those prepared video documents.



The app should not fetch transcripts, split text, and write video documents while a learner is searching. That would make the experience slower, more expensive, and harder to debug.

The rule is simple.

> Do the heavy processing before learners need it. Use the prepared results when learners search.

Also define provider support clearly. If the app supports YouTube, Vimeo, or Bunny, support means both ingestion and playback work. Do not claim a provider is supported if you can ingest captions but cannot seek during playback, or if you can embed playback but cannot create transcript chunks.

---

SECTION 11

## Explain search configuration

Search behavior should not be buried only in code. A learning platform with AI search needs a way to tune what the agent sees and how it behaves.

That is what the Context document is for. It can tell the agent which content types matter, which fields to prioritize, what kind of results to return, and what never to expose.

FOCUS ON

For this project, search should focus on learner facing content such as courses, modules, lessons, lesson notes, key points, learning outcomes, instructors, categories, video chapters, and transcript chunks.

AVOID

It should avoid internal or unsafe content such as private tokens, configuration secrets, full transcript arrays, and raw internal video documents as learner facing results.

The most important search rules should live in both the Context document and the inline system prompt.

That may sound repetitive, but it is useful. If a rule is critical, repeat it where the model is most likely to follow it.

---

SECTION 12

## Define how search must behave

Search is the special feature in this learning platform, so this section needs to be specific.

Search should be a full results page. It should not be a tiny widget, and it should not be a chatbot. The learner should get ranked clickable result cards.

The main result types are video moment result and lesson result.

| VIDEO MOMENT RESULT | LESSON RESULT |
| --- | --- |
| A video moment result is tied to a real lesson video at a specific second. It should include the course name, module and lesson label, thumbnail, short description, matched second, and an action that opens the lesson from that second. | A lesson result is a lesson matched by topic. It should include the course, module and lesson label, lesson key points, short description, and an action to open the lesson. |

- The video document itself should not be shown as a result. The learner should see the lesson, not the internal lookup document.

Search should match both lesson topics and video moments. It should rank specific matches higher than weak matches. A lesson title that directly contains the searched concept should rank higher than a note that mentions a broad keyword once.

> Most importantly, search must be grounded. It must never invent courses, lessons, instructors, URLs, prices, durations, timestamps, or result counts.
>
> `Every result should come from real data.`

---

SECTION 13

## List common traps

This section helps the AI avoid mistakes that are not obvious from the code.

For this learning platform, there are several traps worth writing down.

| | |
|---|---|
| 01 Sanity Context may require a deployed Studio app before the dataset can be served. | 02 Semantic search may not be enabled, so keyword fallback may be needed. |
| 03 Template literal prompts can break if backticks are not escaped. | 04 Search prompt changes may require a server restart. |
| 05 Whole transcript arrays should not be returned because they can overflow context and create noisy results. | 06 The dataset is private, so read tokens must stay server side. |

Also remind the AI which values are safe for the browser and which are not.

CLIENT SAFE

Clerk publishable keys and PostHog public project keys are client safe.

NOT CLIENT SAFE

Clerk secret keys, Sanity tokens, OpenAI keys, and private PostHog keys are not.

This section is useful because the AI may write code that looks correct but fails in the real environment. Practical warnings help it avoid known mistakes.

---

SECTION 14

## Define checks to run

AI should never say something works without checking.

Tell it exactly what to run. For this learning platform, normal checks include lint, type check, production build when routes or server code changed, dev server, and manual browser testing.

For Sanity work, the AI should verify the Studio opens, schemas appear correctly, content can be created, imports work, and Context setup is valid when search depends on it.

For search and ingestion work, it should test with real learner queries, unrelated queries, empty states, no invented results, timestamp behavior, and the real dataset or MCP endpoint when needed.

The rule should be direct.

```
Report the real output. Never claim a check passed without running it.
```

This makes the AI accountable.

---

SECTION 15

## Add a when in doubt section

End the file with simple fallback rules. This gives the AI a reset point when the task is unclear.

For this project, the rules can be written like this.

| | |
|---|---|
| → Keep it small. | → Use the relevant skill. |
| → Preserve server and client boundaries. | → Keep private tokens private. |
| → Match the provided UI exactly. | → Inspect setup and config before hardcoding. |
| → Save a prompt and get approval before coding. | → Run checks. |
| → Share exact test steps. | |

When the AI is unsure, it should not improvise. It should return to these rules.

---

# How to write AGENTS.md for your own project

The easiest way to write a strong `AGENTS.md` is to build it from the project outward.

Do not start with generic agent rules. Start with what the product actually is.

## 1 Describe the product in plain language

Write one clear paragraph.

For this learning platform, that might be this.

> This is a learning platform where authors create courses in Sanity and learners browse courses in a Next.js app, open lessons, watch videos, and search across learning content. The standout feature is intelligent search that returns grounded result cards and can link directly to exact moments in lesson videos.

If you cannot write this paragraph clearly, you are not ready to write the full `AGENTS.md`.

## 2 List the main user flows

For the learning platform, the main flows are straightforward. A learner browses the catalog, opens a course, opens a lesson, watches a video, searches for a topic, opens a search result, and later resumes or completes lessons. An author manages content in Sanity Studio.

These flows tell the AI what the product actually does. They also help decide build order.

## 3 List the tools and why each exists

Do not only list tools. Explain their jobs.

| | |
|---|---|
| For this project, Next.js serves the learner app. | Sanity stores course and lesson content. |
| Sanity Studio is the authoring interface. | Clerk handles authentication. |
| PostHog tracks product behavior. | Sanity Context MCP connects AI search to Sanity content. |
| OpenAI powers search reasoning. | Tailwind handles styling. |
| Zod validates structured output. | |

This prevents the AI from replacing your stack or using the wrong tool for a feature.

## 4 Define the boundaries

This is where the project becomes safe.

For example, the browser never holds private tokens, never calls the LLM directly, never calls the MCP directly, and never writes content or progress directly. Sanity content reads happen server side. Progress writes happen through server routes. Video ingestion runs offline. Search returns structured cards, not chat.

These rules should be written clearly because they prevent dangerous shortcuts.

## 5 Define the data model

Do this before coding.

For this learning platform, define Course, Module, Lesson, Instructor, Category, Video, Agent Context, and Progress. Explain what each one stores and how they relate.

This prevents bad schemas, duplicated relationships, and confusing data access later.

## 6 Define major feature behavior

For important features, write rules directly.

| | |
|---|---|
| SEARCH | For search, say that it must be grounded, return structured cards, support lesson results and video moment results, use chapters before transcript chunks for timestamps, never invent data, and never expose full transcript chunks. |
| VIDEO | For video, say that playback stays on the site, uses provider embeds, does not use a custom video player, and uses a start seconds query param for deep links. |
| ANALYTICS | For analytics, say that PostHog tracks meaningful learning events and must not receive private tokens or sensitive data. |

### 7 Define the AI workflow

This is where you make the AI disciplined.

Tell it not to code immediately. It should read `AGENTS.md`, read relevant skills, inspect existing code, create an implementation prompt in `prompts`, ask for approval, build only after approval, run checks, and report with `What I did`, `Test`, and `Needs your attention`.

This turns random prompting into a repeatable workflow.

### 8 Define checks and manual tests

Do not leave testing vague.

Tell the AI to run lint, build, typecheck if available, open the app in the browser, test the happy path, test empty states, test invalid inputs, check Studio when Sanity changes, check PostHog when analytics changes, and test search with real and unrelated queries.

This makes done measurable.

---

## A strong AGENTS.md structure

For a project like this, use this structure.

AGENTS.md

```
# AGENTS.md

You are a role building a product.

Your job is to follow the project workflow.

## 1. What you are building

Explain the product, users, core value, and scope.

## 2. How to work

Explain the AI workflow.

## 3. UI work

Explain how UI references should be handled.

## 4. Skills and docs to use

List project skills, local docs, and package docs.

## 5. App structure and boundaries

Explain what belongs where and what must stay server side.

## 6. Tech stack

List the stack and what not to use.

## 7. Decisions already made

Record product and architecture decisions the AI must not decide again.
```

AGENTS.md — continued

```
## 8. Data model

Define every major document, table, or record.

## 9. Background or offline processes

Explain ingestion, scripts, sync, or processing that should not run during
requests.

## 10. Config and tuning

Explain documents or settings that tune behavior without code changes.

## 11. Feature behavior

Define the most important feature rules in detail.

## 12. Things that will trip you up

List practical warnings and common mistakes.

## 13. Checks to run

Tell the AI what to run and what to verify.

## 14. When in doubt

Give simple fallback rules.
```

You can modify or come up with your own structure too as long as you cover everything nicely.

## The biggest mistake to avoid

The biggest mistake is writing `AGENTS.md` as vague motivation.

Bad version.

```
- Build clean code.
- Use best practices.
- Make the UI nice.
- Keep things secure.
```

This sounds good, but it does not help the AI much. It does not explain what clean means, which practices matter, what the UI should match, or what secure means in this project.

Better version.

```
✓ Use the existing Tailwind and component patterns before adding new ones.
✓ Match the provided desktop UI reference exactly.
✓ Keep the Sanity read token server side.
✓ The browser must never call the MCP or LLM.
✓ Search returns structured result cards, not chat.
✓ Modules are embedded inside courses, not documents.
```

Specific beats inspirational. The AI needs rules it can act on.

## How detailed should AGENTS.md be

`AGENTS.md` should be detailed enough that the AI does not need to make product or architecture decisions during implementation, but not so bloated that the important rules are buried.

For a small weekend app, a shorter file may be enough. For this learning platform, a detailed file makes sense because the app has several connected systems. Next.js pages, Sanity Studio, Clerk auth, PostHog analytics, AI search, video ingestion, timestamp playback, progress tracking, server routes, and environment variables all need clear rules.

```
The test is simple. If the AI could make a dangerous or
product breaking assumption, write the rule down.
```

---

## When to update AGENTS.md

Do not treat `AGENTS.md` as frozen forever. Update it when a major decision changes.

For example, update it if the route structure changes, the auth model changes, the search strategy changes, the data model changes, a new video provider is added, or a common AI mistake keeps happening.

But do not update it for every tiny implementation detail. Stable project rules belong in `AGENTS.md`. Feature specific details belong in implementation prompt files inside `prompts`.

This keeps the AGENTS file useful instead of turning it into a noisy project diary.

---

## A prompt to create AGENTS.md for this project

Use this with a planning AI.

planning prompt

```
I am building a learning platform.

Authors create courses in Sanity. Learners browse courses in a Next.js
app, open course pages, watch lesson videos, and search across course
and lesson content.

The standout feature is intelligent search. Learners type plain language
questions and get grounded result cards that link to real courses,
lessons, or exact moments in lesson videos.

Help me draft an AGENTS.md file for an AI coding agent.

Include the role of the AI agent, what the app is, how the AI should
work, UI implementation rules, skills and docs it should use, app
responsibilities and boundaries, tech stack, decisions already made,
data model, video ingestion rules, search behavior, common pitfalls,
checks to run, and when in doubt rules.

Write it as direct instructions to the agent. Be specific and concrete.
Avoid vague advice.
```

Then review the output carefully.

Ask yourself whether every section is true for your project. Check whether anything is missing, too broad, or unsafe. Make sure it says what not to do. Make sure it protects private values. Make sure it keeps scope clear. Make sure it explains search well enough. Make sure it matches the actual build order.

Do not accept the first draft blindly. The AI can help write the file, but you are responsible for the decisions inside it.

# Takeaway

## `AGENTS.md` is how you give the AI a real system to work inside.

It gives the agent product context, technical context, workflow rules, security rules, data model, feature behavior, testing expectations, and scope boundaries.

For this learning platform, that matters because the product has many connected pieces.

- The catalog depends on Sanity.
- The course page depends on the content model.
- The lesson page depends on lessons and video URLs.
- Search depends on Sanity Context, schemas, video documents, and grounding rules.
- Timestamp playback depends on transcript chunks and chapter markers.
- PostHog depends on meaningful product events.

All of that needs a shared source of truth. That is what `AGENTS.md` gives you.

# Final Thoughts

And this is also where the bigger workflow starts to matter.

In this video, we are using one `AGENTS.md` file to give the AI strong project context. That is enough for a focused build like this learning platform, where we want to move fast and keep the process simple.

But when you start building larger products, this idea expands. Instead of one file, you can split the project context into multiple focused documents. One file explains the product, one explains the architecture, one explains the AI workflow, one tracks progress, one documents UI rules, and one keeps the standards clear.

### That is the deeper Agentic Engineering workflow.

If you enjoyed this process and you want to go beyond a single tutorial build, that is what we cover inside the Agentic Engineering course. We go much deeper into how to scope a product, structure context files, plan features, guide AI agents, review their work, keep long projects on track, and build production style apps with a repeatable system.

So think of this lesson as the first version of the workflow. `AGENTS.md` gives the AI a strong starting point for this project.

The full Agentic Engineering process teaches you how to scale that same idea across bigger products, longer builds, and more serious codebases.